INGENIERÍA EN SOFTWARE

Testing aplicado

Bloque 02 · Apunte completo de estudio

Pirámide de pruebas · xUnit en C# · Patrón AAA
Mocks y Stubs · Cobertura · TDD

Instituto Técnico Superior Córdoba

Ingeniería en Software · Prof. Mateo Alexis Vranich Prieto · Córdoba 2026

#	Sección
01	¿Qué es el testing?
02	Pirámide de pruebas
03	Anatomía de un test: patrón AAA
04	xUnit en C#
05	Mocks, Stubs y Fakes
06	Cobertura de código
07	TDD: Test-Driven Development

01 · ¿Qué es el testing?

¿Por qué testear?

El testing es el proceso de verificar que el software hace lo que debe hacer y detectar errores antes de que lleguen a producción. Un sistema sin tests es como una lomitería que no prueba los ingredientes antes de cocinar: puede salir bien, pero cuando algo falla, el cliente se entera primero que vos.

Testear no es opcional en ingeniería de software. Es lo que separa "el código funciona" de "el código es correcto". Los bugs encontrados en producción cuestan entre 10x y 100x más que los encontrados durante el desarrollo.

Verificación vs Validación

Concepto	Pregunta que responde	Ejemplo lomitería
Verificación	¿Estamos construyendo el sistema correctamente?	¿El lomito tiene los ingredientes que dice la receta?
Validación	¿Estamos construyendo el sistema correcto?	¿El lomito es lo que el cliente realmente quería?

Tipos de errores

Tipo	Descripción	Ejemplo
Error (mistake)	Equivocación humana del programador.	El dev olvidó validar que el precio sea positivo.
Defecto (bug)	El error se materializa en el código.	La función acepta precios negativos.
Fallo (failure)	El defecto se manifiesta en ejecución.	Un cliente compra un lomito a -$500 y le pagan a él.

02 · Pirámide de pruebas

Las pruebas se organizan en una pirámide propuesta por Mike Cohn. La base tiene muchas pruebas unitarias (rápidas y baratas), la capa media tiene menos pruebas de integración, y la punta tiene muy pocas pruebas end-to-end (lentas y costosas). El objetivo es detectar errores lo más temprano y barato posible.

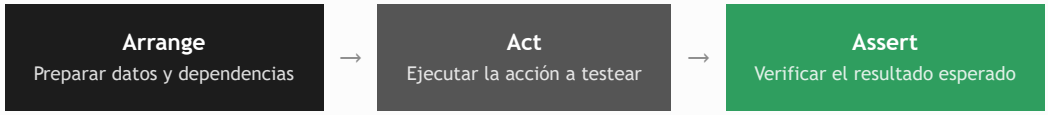


Muchas pruebas rápidas en la base; pocas pruebas lentas en la cima.

Tipo	¿Qué prueba?	Velocidad	Ejemplo lomitería
Unitaria	Una función o clase aislada.	Muy rápida	Probar que CalcularPrecio() suma bien los ingredientes.
Integración	Interacción entre componentes.	Media	Probar que el servicio de pedidos graba en la base de datos.
Sistema (E2E)	El sistema completo end-to-end.	Lenta	Simular un cliente que pide, paga y recibe su lomito.
Aceptación (UAT)	Que cumple los requisitos del cliente.	Variable	El dueño prueba que el sistema muestra sus reportes de ventas.
Regresión	Que cambios nuevos no rompieron lo existente.	Variable	Agregar delivery no rompió el flujo de mostrador.

03 · Anatomía de un test: patrón AAA

Todo test bien escrito sigue el patrón **AAA (Arrange-Act-Assert)**. Es la estructura universal que hace los tests legibles, predecibles y fáciles de mantener. Pensalo como preparar un lomito: primero preparás los ingredientes (Arrange), después cocinás (Act), y finalmente probás que quedó bien (Assert).



Fase	¿Qué hace?	En el test
Arrange	Prepara los datos, objetos y dependencias necesarias.	Crear el objeto Lomito, configurar el precio, instanciar el servicio.
Act	Ejecuta la acción que querés testear. Una sola línea idealmente.	Llamar a CalcularPrecioFinal().
Assert	Verifica que el resultado es el esperado.	Assert.Equal(1500, resultado).

Ejemplo completo en C# con xUnit

xUnit · [Fact]

```
[Fact]
public void CalcularPrecioFinal_ConIngredientesExtra_SumaCorrectamente()
{
    // Arrange
    var lomito = new Lomito("Completo", precioBase: 1200);
    lomito.AgregarIngrediente("Huevo frito", 300);

    // Act
    var precioFinal = lomito.CalcularPrecioFinal();

    // Assert
    Assert.Equal(1500, precioFinal);
}
```

> **Tip:** El nombre del test sigue la convención *Método_Escenario_ResultadoEsperado*. No uses nombres como "Test1" o "ProbarCosa": un buen nombre es documentación.

04 · xUnit en C#

xUnit es el framework de testing más usado en el ecosistema .NET moderno. Es el sucesor espiritual de NUnit, creado por los mismos autores. Se integra nativamente con Visual Studio y dotnet CLI.

Configuración inicial

terminal

```
dotnet new xunit -n MiApp.Tests
cd MiApp.Tests
dotnet add reference ../MiApp/MiApp.csproj
dotnet test
```

[Fact] vs [Theory]

Atributo	¿Cuándo usarlo?	Ejemplo
[Fact]	Test con un solo escenario fijo. No recibe parámetros.	Verificar que un lomito sin extras cuesta el precio base.
[Theory]	Test parametrizado: se ejecuta múltiples veces con distintos datos.	Probar descuentos con 5%, 10%, 20% usando [InlineData].

Ejemplo con [Theory] e [InlineData]

xUnit · [Theory]

```
[Theory]
[InlineData(1000, 10, 900)]  // 10% de descuento
[InlineData(1000, 20, 800)]  // 20% de descuento
[InlineData(500, 0, 500)]    // sin descuento
public void AplicarDescuento_ConDistintosPorcentajes_CalculaBien(
    decimal precio, int porcentaje, decimal esperado)
{
    var servicio = new PrecioService();
    var resultado = servicio.AplicarDescuento(precio, porcentaje);
    Assert.Equal(esperado, resultado);
}
```

Asserts más usados

Assert	¿Qué verifica?	Ejemplo
Assert.Equal(a, b)	Que dos valores son iguales.	Assert.Equal(1500, precio)

Assert	¿Qué verifica?	Ejemplo
Assert.True(cond)	Que la condición es verdadera.	Assert.True(lomito.EsAptoVegano())
Assert.Null(obj)	Que el objeto es null.	Assert.Null(repo.Buscar(999))
Assert.NotEmpty(list)	Que la lista no está vacía.	Assert.NotEmpty(pedidos)
Assert.Throws<T>()	Que se lanza una excepción específica.	Assert.Throws<ArgumentException>(() => ...)
Assert.Contains(item, list)	Que la colección contiene el elemento.	Assert.Contains("Huevo", ingredientes)

05 · Mocks, Stubs y Fakes

Cuando una clase depende de otra (base de datos, API externa, servicio de email), no querés que el test unitario dependa de esas cosas reales. Usás **dobles de prueba** (test doubles) para reemplazarlas con versiones controladas.

Es como ensayar una obra de teatro: no necesitás al público real para practicar tus líneas. Usás compañeros que hacen de público.

Doble	¿Qué hace?	Ejemplo lomitería
Stub	Devuelve datos predefinidos. No verifica que se lo llame.	Un StubRepo que siempre devuelve los mismos 3 lomitos sin ir a la BD.
Mock	Verifica que se llamó un método específico con parámetros esperados.	Verificar que al confirmar pedido se llamó a EnviarNotificacion().
Fake	Implementación simplificada pero funcional.	Un repositorio en memoria (List<Lomito>) en lugar de SQL Server.
Spy	Como un stub pero registra las llamadas para verificar después.	Registrar cuántas veces se llamó a CobrarPedido().

Ejemplo práctico: Stub manual

Stub

```
// Interfaz real
public interface ILomitoRepo {
    List<Lomito> ObtenerTodos();
    Lomito ObtenerPorId(int id);
}

// Stub para tests: siempre devuelve datos predefinidos
public class StubLomitoRepo : ILomitoRepo {
    public List<Lomito> ObtenerTodos() => new List<Lomito> {
        new Lomito("Completo", 1500),
        new Lomito("Árabe", 1800),
    };
    public Lomito ObtenerPorId(int id) => new Lomito("Test", 1000);
}
```

Ejemplo práctico: Mock con Moq

Mock · Moq

```
[Fact]
public void ConfirmarPedido_EnviaNotificacion()
{
    // Arrange
    var mockNotif = new Mock<INotificador>();
    var servicio = new PedidoService(mockNotif.Object);
    var pedido = new Pedido("Lomito completo", 1500);

    // Act
    servicio.Confirmar(pedido);

    // Assert: verificar que se llamó al método
    mockNotif.Verify(
        n => n.Enviar(It.IsAny<string>()),
        Times.Once
    );
}
```

> **Tip:** ¿Cuándo usar cada uno? Si solo necesitás datos de prueba, usá un **Stub**. Si necesitás verificar que se llamó a un método, usá un **Mock**. Si necesitás lógica simplificada, usá un **Fake**.

06 · Cobertura de código

La cobertura de código mide qué porcentaje del código fuente se ejecuta cuando corren los tests. Es una métrica útil pero engañosa: 100% de cobertura no significa 0 bugs. Podrías ejecutar cada línea sin verificar nada útil.

Tipo de cobertura	¿Qué mide?	Ejemplo
Líneas (Statement)	% de líneas de código ejecutadas.	80% = 80 de 100 líneas se ejecutaron.
Ramas (Branch)	% de caminos en if/else/switch ejecutados.	Un if con solo el true testeado = 50% branch.
Funciones	% de funciones/métodos llamados.	10 de 12 métodos testeados = 83%.

> **Tip:** Un objetivo razonable es 70-80% de cobertura de líneas. Más importante que el número es testear la lógica crítica: cálculos de precios, validaciones, flujos de pago. Un getter trivial no necesita test.

Cómo medir cobertura con dotnet

terminal

```
# Instalar la herramienta de reportes
dotnet tool install -g dotnet-reportgenerator-globaltool

# Correr tests con cobertura
dotnet test --collect:"XPlat Code Coverage"

# Generar reporte HTML
reportgenerator -reports:coverage.cobertura.xml \
    -targetdir:CoverageReport -reporttypes:Html
```

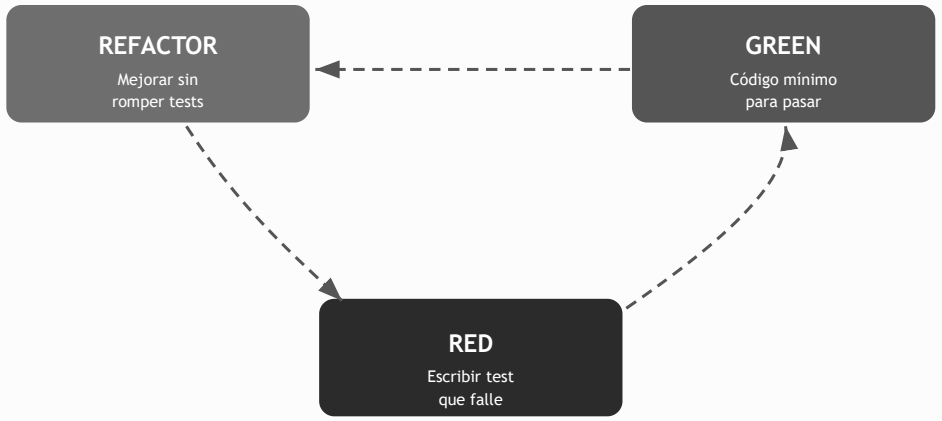
¿Qué testear y qué no?

Prioridad alta (testear siempre)	Prioridad baja (opcional)
Lógica de negocio (cálculos, reglas, validaciones) · Flujos críticos (pagos, autenticación) · Casos borde (listas vacías, valores nulos, límites) · Bugs encontrados (test de regresión)	Getters y setters triviales · Código generado automáticamente · Configuración de infraestructura · Código de UI sin lógica

07 · TDD: Test-Driven Development

TDD es una técnica de desarrollo donde se escribe el **test antes que el código**. Suena contraintuitivo, pero la idea es poderosa: si definimos primero qué esperamos del código, el código se escribe con un propósito claro.

El ciclo Red-Green-Refactor



Ciclo corto y repetido: test que falla → código mínimo → mejorar estructura.

Paso	Color	¿Qué hacés?	Ejemplo
1	**RED**	Escribir un test que falle. El código que testea aún no existe.	Test: CalcularTotal() debe devolver 1500 para un lomito completo.
2	**GREEN**	Escribir el código mínimo para que el test pase. Nada más.	Implementar CalcularTotal() lo más simple posible.
3	**REFACTOR**	Mejorar el código sin cambiar su comportamiento. Los tests siguen pasando.	Extraer métodos, eliminar duplicación, mejorar nombres.

Ejemplo TDD paso a paso

Queremos implementar un método que calcule si un pedido tiene envío gratis (pedidos mayores a $5000).

Paso 1 — RED: escribimos el test primero

RED

```
[Fact]
public void TieneEnvioGratis_PedidoMayorA5000_DevuelveTrue()
{
    var pedido = new Pedido { Total = 6000 };
    var resultado = pedido.TieneEnvioGratis();
    Assert.True(resultado);
} // ← No compila: TieneEnvioGratis() no existe aún
```

Paso 2 — GREEN: código mínimo para que pase

GREEN

```
public bool TieneEnvioGratis() => Total > 5000;
```

Paso 3 — REFACTOR: extraer el umbral como constante

REFACTOR

```
private const decimal UmbralEnvioGratis = 5000;
public bool TieneEnvioGratis() => Total > UmbralEnvioGratis;
```

> **Tip:** TDD no significa escribir todos los tests primero. Se hace de a uno: un test, código mínimo, refactor, repetir. Es un ciclo corto y rápido.

08 · Buenas prácticas, errores comunes y glosario

Los tests son código de primera clase

Los tests no son código descartable: se mantienen, se refactorizan y se revisan igual que el código de producción. Un test ilegible es peor que no tener test, porque da falsa seguridad.

Práctica	Descripción	Ejemplo
Un assert por test	Cada test verifica una sola cosa. Si falla, sabés exactamente qué se rompió.	No mezclar Assert.Equal del precio con Assert.True del stock.
Nombres descriptivos	Método_Escenario_Resultado. El nombre es documentación.	AplicarDescuento_PorcentajeNegativo_LanzaExcepcion
Tests independientes	Cada test puede correr solo, sin depender de otros ni de un orden.	No usar variables estáticas compartidas entre tests.
Tests deterministas	El mismo test con los mismos datos siempre da el mismo resultado.	No depender de DateTime.Now ni de Random sin seed.
Tests rápidos	Los unitarios deben correr en milisegundos. Si son lentos, nadie los corre.	Evitar acceso a BD, red o filesystem en tests unitarios.
Testear el comportamiento	Verificar qué hace el código, no cómo lo hace internamente.	Testear que el precio es correcto, no que se llamó a un método privado.

Errores comunes en testing

- **Testear métodos privados directamente.** Los métodos privados se testean indirectamente a través de los públicos que los usan. Si sentís que necesitás testear un privado, probablemente debería ser una clase aparte.
- **Tests que dependen del orden de ejecución.** Si el test B falla solo cuando corre después del test A, hay estado compartido. Cada test debe poder correr aislado.
- **Asserts sin sentido.** Assert.True(true) o Assert.Equal(x, x) no testean nada. El assert debe verificar algo que podría fallar.
- **Tests que testean el framework.** No testees que List.Add() agrega un elemento: eso ya lo testea Microsoft. Testeá tu lógica de negocio.
- **Ignorar tests que fallan.** Un test rojo ignorado es peor que no tener test. Si falla, se arregla o se borra. Nunca se deja en [Skip] indefinidamente.
- **Confundir unitarios con integración.** Si tu test "unitario" necesita una base de datos corriendo, es de integración. Usá stubs o mocks para mantener los unitarios aislados y rápidos.

Glosario del bloque

Término	Definición
Test unitario	Prueba aislada de una función o clase, sin dependencias externas.
Test de integración	Prueba que verifica la interacción entre dos o más componentes.
Test E2E	Prueba del sistema completo, simulando el uso real del usuario.
AAA	Arrange-Act-Assert. Patrón universal para estructurar tests.
Mock	Doble de prueba que verifica que se llamó a un método específico.
Stub	Doble de prueba que devuelve datos predefinidos sin verificar llamadas.
Fake	Implementación simplificada pero funcional de una dependencia.
Cobertura	Porcentaje de código ejecutado por los tests.
TDD	Test-Driven Development. Técnica de escribir el test antes que el código.
Red-Green-Refactor	Ciclo de TDD: test que falle, código mínimo, mejorar estructura.
Regresión	Test que verifica que cambios nuevos no rompieron funcionalidad existente.
[Fact]	Atributo de xUnit para un test con un solo escenario fijo.
[Theory]	Atributo de xUnit para tests parametrizados con múltiples datos.
Moq	Librería de .NET para crear mocks y stubs de forma dinámica.